

C4: Sorting and Recursion

Name: Alston Liow Guan You

Student ID: 37440985

Date: 24 August 2026

Overview

Explore recursion and ideas of algorithm efficiency by creating a program that visualises sort programs in action. Extend your glossary with these new concepts.

Focus on understanding the implementation

Here the focus is on the implementation, not the details of how the algorithm works. This should help you see that you can implement an algorithm, even without knowing how it works.

Designers of new algorithms need to prove that their algorithm achieves its stated goals. This is why Computer Science will focus on mathematical proofs - something you should pay attention to if you are studying computer science. Everyone else can then use the algorithm by implementing it in code.

Do the following:

Part 1: Simple iterative sort

- You want to demonstrate that you can implement some sort algorithms, and display these graphically. You can use the [Sort Visualiser](#) guided tour (including extensions) to get bubble sort working.
- Select one of the following sorting algorithms:
 - Insertion sort
 - Selection sort
- Implement the algorithm yourself using the algorithm's details from the end of this document.

Coding convention

The details at the end of this document are taken from prompts given to a GenAI. We are giving you exactly what it responded with. Make sure to apply our coding convention when you implement this!

- Add in calls to visualise your chosen algorithm's actions, ensuring that you are calling `visualise` for each comparison within the algorithm.
- Allow the user to choose between your chosen algorithm and bubble sort when they run your program.
- Update your glossary with a page on your sorting algorithm. Use this to capture your observations of how it works.

Communicate your thoughts - no need to research this in depth

Remember the focus is on your implementation of these algorithms. We would like *your* thoughts on how these work - you can learn the theory and other details in later studies if that is relevant to you. So, for example, don't tell us about $O(n^2)$!

Part 2: Recursive sort

- Review the topic of [recursion](#) in the Field Guide.
- Add a page to your glossary on recursion and how it works.
 - Ensure you have a visualisation or metaphor for how recursion works
- Now, select one of the following sorting algorithms:
 - Quick sort
 - Merge sort
- Use the pseudocode from the end of this document, and implement this with calls to `visualise` the array as it runs.
- Allow the user to choose between your chosen algorithms and bubble sort when they run your program.
- Update your glossary with a page on your new sorting algorithm. Use this to capture your observations of how it works.

Reflect:

- What is the most important thing you learned from this and why?

> I learned about sort algorithm with directly and recursion. For each algorithm I understand by draw out the procedure and also watching YouTube visualise video. Know I can sort a array of number using a suitable algorithm with most efficiency.

- How did you go with understanding recursion?

> Recursion is the function repeating using itself until reach the initial case. This can save the time. Recursion is not look so abstract but when go deeply with some application or some algorithm, it could be very difficult to understand combine with the algorithm. For example is merge sort, initially I don't know why it need to split to the smallest group until I see that it can group and sort in same time using the double pointer.

- From your observations, which algorithm was fastest? Why do you think that is?

> I think merge sort and quick sort is fastest, just depend on the array we use. By recursive, merge sort can split the large group into smallest group. So the operation will be very fast. Bubble sort is always looping to compare the neighbour element, insertion sort also have double loop.

- Looking back, how long did it take to complete this task? What did you spend the most time on?

> I take 16 hours. Actually I spend the time most in understanding the algorithm and thinking about how to split the algorithm to few function to combine with the visualise procedure.

Disclose:

- Explain your use of Generative AI for this task. If you didn't use it, just write "I did not use any AI for this task". If you did use it, please state which AI model you used (e.g. ChatGPT) and how.

> I use Claude to understanding all sort algorithm.

Submit:

- Export this worksheet as a PDF.

- Take a photograph of two pages from your glossary, one that is representative of your work in general, and one that you are particularly proud of or feel was particularly influential in your learning.
- Upload your submission to OnTrack.
- Engage with feedback and discussion to get this signed off.

Insertion Sort - from ChatGPT

Provide me with a simple algorithmic description of insertion sort

Insertion sort works like sorting playing cards in your hand.

Start with the second item. Compare it to the items before it, and move it left until it is in the correct position. Then take the next item and do the same. Repeat until the whole list is sorted.

Algorithmically:

1. Assume the first item is already sorted.
2. Take the next item.
3. Move it left while it is smaller than the items before it.
4. Insert it into the correct spot.
5. Repeat for every remaining item.

Pseudocode:

```
for i from 1 to n-1:
    key = A[i]
    j = i - 1
    while j >= 0 and A[j] > key:
        A[j + 1] = A[j]
        j = j - 1
    A[j + 1] = key
```

It is simple and works well for small or nearly sorted lists.

Selection Sort - from ChatGPT

Provide me with a simple algorithmic description of selection sort

Selection sort is like repeatedly picking the smallest remaining item and putting it in place.

Start at the beginning of the list. Find the smallest item in the unsorted part, swap it with the first unsorted position, then move one step to the right and repeat. Continue until the whole list is sorted.

Algorithmically:

1. Treat the left part of the list as sorted.
2. Find the smallest item in the unsorted part.
3. Swap it with the first item in the unsorted part.
4. Move the boundary of the sorted part one position right.
5. Repeat until done.

Pseudocode:

```
for i from 0 to n-2:
    min_index = i
    for j from i+1 to n-1:
        if A[j] < A[min_index]:
            min_index = j
    swap A[i] and A[min_index]
```

It is easy to understand, but not efficient for large lists.

Quick Sort - from ChatGPT

Provide a simple algorithmic description of quick sort, include pseudocode for all functions, and sort the array in place.

Quick sort works by choosing a pivot, arranging the array so that smaller items go to the left of the pivot and larger items go to the right, then repeating the same process on the two smaller parts. The array is sorted in place by swapping elements inside the original array.

Algorithmically:

1. Choose a pivot element.
2. Partition the array so everything smaller than the pivot is moved to the left, and everything larger is moved to the right.
3. Put the pivot into its final position.
4. Recursively quick sort the left part.
5. Recursively quick sort the right part.
6. Stop when a part has zero or one item.

Pseudocode:

```
quickSort(A, low, high):  
    if low < high:  
        p = partition(A, low, high)  
        quickSort(A, low, p - 1)  
        quickSort(A, p + 1, high)
```

```
partition(A, low, high):  
    pivot = A[high]  
    i = low - 1  
    for j from low to high - 1:  
        if A[j] <= pivot:  
            i = i + 1  
            swap(A[i], A[j])  
    swap(A[i + 1], A[high])  
    return i + 1
```

```
swap(x, y):  
    temp = x  
    x = y  
    y = temp
```

Merge Sort - from ChatGPT

Provide me with a simple algorithmic description of merge sort, include pseudocode for all functions, and sort the array in place.

Merge sort works by splitting the array into smaller pieces, sorting those pieces, and then merging them back into the same array.

Algorithmically:

1. Split the array into two halves.
2. Recursively sort the left half.
3. Recursively sort the right half.
4. Merge the two sorted halves back into the original array.
5. Repeat until the full array is sorted.

Pseudocode:

```
mergeSort(A, left, right):  
    if left >= right:  
        return  
  
    mid = (left + right) / 2  
    mergeSort(A, left, mid)  
    mergeSort(A, mid + 1, right)  
    merge(A, left, mid, right)
```

Merge pseudocode:

```
merge(A, left, mid, right):  
    temp = empty array  
    i = left  
    j = mid + 1  
  
    while i <= mid and j <= right:  
        if A[i] <= A[j]:  
            append A[i] to temp  
            i = i + 1  
        else:  
            append A[j] to temp  
            j = j + 1  
  
    while i <= mid:  
        append A[i] to temp  
        i = i + 1  
  
    while j <= right:  
        append A[j] to temp  
        j = j + 1
```



```
for k from 0 to length of temp - 1:  
    A[left + k] = temp[k]
```

Merge sort is efficient for large lists because it divides the problem into smaller parts before combining them.